

Package ‘adatutor’

July 25, 2025

Title Introducing the Boosting Framework with AdaBoost

Version 1.0.0

Description The adatutor package supports our efforts to introduce boosting to psychologists with little machine learning background. It provides the necessary datasets and some convenience functions that we use in our hands-on tutorial.

License GPL (>= 3)

Encoding UTF-8

Roxygen list(markdown = TRUE)

RoxygenNote 7.3.2

Depends R (>= 4.5)

Imports utils (>= 4.5),
stats (>= 4.5),
AUC (>= 0.3),
rpart (>= 4.1),
rpart.plot (>= 3.1),
rsample (>= 1.2)

LazyData true

Suggests knitr,
rmarkdown,
testthat (>= 3.0.0)

VignetteBuilder knitr

Config/testthat/edition 3

URL <https://sbissantz.github.io/adatutor/>

Contents

altmejd	2
altmejd723	2
checks	3
feedback	5
random_split	6
ssrp723	7
stratified_split	7
testAda	8
trainAda	9

Index	11
--------------	-----------

altmejd

*Short version of Altmejd et al.'s replication database***Description**

An adapted version of the replication database by Altmejd et al. (2018). It includes replication projects of social science lab experiments for five large-scale replication projects: Many Labs 1 and 3, the Replication Project Psychology, the Experimental Economics Replication Project, and the Social Science Replication Project.

Usage

altmejd

Format

A data frame with 152 study effects (i.e., rows) rows; 2 identifiers, and 5 predictors (i.e., 7 columns):

eid Effect identifier

pid Project identifier: Many Labs 1 (ML1), Many Labs 3 (ML3), Replication Project Psychology (RPP), Experimental Economics Replication Project (EERP), Social Science Replication Project (SSRP)

effect_size.o Standardized effect size of the original study

n.o Number of observations in the original study

p_value.o P value of the original study

power.o Power of the original study

replicate Replication success indicator

Source

Altmejd A, Dreber A, Forsell E, Huber J, Imai T, et al. (2019) Predicting the replicability of social science lab experiments. PLOS ONE 14(12): e0225826. <https://doi.org/10.1371/journal.pone.0225826>

The data and additional information on the data set can be downloaded from their OSF repository. <https://osf.io/4fn73/>

altmejd723

*An even shorter version of Altmejd et al.'s replication database***Description**

A subset of Altmejd et al.'s (2018) replication database. Note that this dataset is only used internally, to build the full database (see [altmejd](#)). It includes replication projects of social science lab experiments for four large-scale replication projects: Many Labs 1 and 3, the Replication Project Psychology, and the Experimental Economics Replication Project. The data for the Social Science Replication Project (SSRP) are missing.

Usage

```
altmejd723
```

Format

A data frame with 131 study effects (i.e., rows); 2 identifiers, and 5 predictors (i.e., 7 columns):

eid Effect identifier

pid Project identifier: Many Labs 1 (ML1), Many Labs 3 (ML3), Replication Project Psychology (RPP), Experimental Economics Replication Project (EERP)

effect_size.o Standardized effect size of the original study

n.o Number of observations in the original study

p_value.o P value of the original study

power.o Power of the original study

replicate Replication success indicator

Source

Altmejd A, Dreber A, Forsell E, Huber J, Imai T, et al. (2019) Predicting the replicability of social science lab experiments. PLOS ONE 14(12): e0225826. <https://doi.org/10.1371/journal.pone.0225826>

The data and additional information on the data set can be downloaded from their OSF repository. <https://osf.io/4fn73/>

 checks

Miscellaneous Functions for Input Validation

Description

A set of input validation functions for internal use that ensure the passed arguments meet the expected criteria (e.g., type, length, and value ranges). These functions catch common input errors, provide meaningful error messages and guidance for correction.

`check_df()` ensures that `x` is a data frame. If it is not, an error is thrown with a message indicating the issue.

`check_eta()` validates that the input is a numeric value and, if it is greater than 1, provides a warning about potential overfitting.

`check_length()` ensures that `x` contains at least one element.

`check_list()` validates that the input is a list.

`check_numeric()` validates that the input is a numeric value.

`check_train()` ensures that the training and test set names are not the same.

`check_prop()` validates that the input is a proportion (a numeric value between 0 and 1).

Usage

```
check_df(x)

check_eta(x)

check_length(x)

check_list(x)

check_numeric(x)

check_train(trainme, testme)

check_prop(x)
```

Arguments

x	A generic input to be validated. The expected type depends on the specific function.
trainme	A character string representing the name of the training set.
testme	A character string representing the name of the test set.

Details

Ensures that the input is not empty. If the input has a length of 0, an error message is raised with a suggestion to check the input.

Ensures that the input is a valid list. If not, an error message is raised.

Ensures that the input is numeric. If not, an error message is raised with a suggestion to validate the input.

Value

All functions are called for their side effects. If the input is invalid, an error, or warning is generated with helpful suggestions for troubleshooting.

Examples

```
# Example usage:
## Not run: check_df(5) # Fails with an error message

# Example usage:
## Not run: check_eta("a") # Fails with an error

# Example usage:
## Not run: check_length(character(0)) # Fails with an error

# Example usage:
## Not run: check_list(5) # Fails with an error

# Example usage:
## Not run: check_numeric("a") # Fails with an error

# Example usage:
```

```
## Not run: check_train("train", "train") # Fails with an error

# Example usage:
## Not run: check_prop(0) # Issues a warning
```

feedback

Utility Functions for Console Feedback and Styling

Description

A set of utility functions to enhance the console feedback during processes. These functions include animated feedback, colored messages, and combinations of both for a better user experience in the terminal.

`walking_dots()` creates a sequence of animated dots (e.g., "...") followed by a "Done" message. The number of dots and the delay can be customized.

`color_message()` prints a message in a specified color using ANSI color codes. The text and color can be customized.

`walking_colordots()` is a combination of `walking_dots()` and `color_message()`. It prints animated dots in a specified color, with a delay between each, and ends with a colored "Done" message.

Usage

```
walking_dots(n = 3, delay = 0.2)

color_message(text, color_code = 32, newline = FALSE)

walking_colordots(n = 3, delay = 0.2, color_code = 32)
```

Arguments

<code>n</code>	An integer specifying the number of iterations for dots or animations. Defaults to 3.
<code>delay</code>	A numeric value specifying the delay (in seconds) between iterations or dots. Defaults to 0.2.
<code>text</code>	A character string representing the message to be displayed. Used in <code>color_message()</code> .
<code>color_code</code>	An integer specifying the ANSI color code for the text. Defaults to 32 (green).
<code>newline</code>	A logical value indicating whether to append a newline after the message. Defaults to FALSE.

Value

All functions are called for their side effects. They do not return values, but produce visual effects in the console, such as animations or colored text.

Examples

```
# Example usage:
## Not run: walking_dots(n = 5, delay = 0.1)

# Example usage:
## Not run: color_message("A green message", color_code = 32)

# Example usage:
## Not run: walking_colordots(n = 4, delay = 0.15, color_code = 34)
```

random_split

*Randomly Split a Dataset into Two Subsets***Description**

The function takes a dataset and splits it into two subsets randomly, based on a specified proportion.

Usage

```
random_split(set, prop = 0.7)
```

Arguments

set	A data frame or a matrix to be split into subsets.
prop	A numeric value between 0 and 1 indicating the proportion of rows to include in the first subset. Defaults to 0.7.

Value

A list with two elements:

set1 The first subset of the dataset, containing prop proportion of the rows.

set2 The second subset of the dataset, containing the remaining rows.

Examples

```
# Load a dataset
data(altmejd)

# A 70-30 simple random split
split <- random_split(altmejd, prop = 0.7)

# Access the subsets
set1 <- split$set1
set2 <- split$set2
```

ssrp723

*An even shorter version of Altmejd et al.'s replication database***Description**

A subset of Altmejd et al.'s (2018) replication database. Note that this dataset is only used internally, to build the full database (see [altmejd](#)). It includes replication projects of social science lab experiments for just one of the five large-scale replication projects: the Social Science Replication Project (SSRP).

Usage

ssrp723

Format

A data frame with 21 study effects (i.e., rows); 2 identifiers, and 5 predictors (i.e., 7 columns):

eid Effect identifier

pid Project identifier: Social Science Replication Project (SSRP)

effect_size.o Standardized effect size of the original study

n.o Number of observations in the original study

p_value.o P value of the original study

power.o Power of the original study

replicate Replication success indicator

Source

Altmejd A, Dreber A, Forsell E, Huber J, Imai T, et al. (2019) Predicting the replicability of social science lab experiments. PLOS ONE 14(12): e0225826. <https://doi.org/10.1371/journal.pone.0225826>

The data and additional information on the data set can be downloaded from their OSF repository. <https://osf.io/4fn73/>

stratified_split

*Stratified Split of a Dataset into Two Subsets***Description**

The function takes a dataset and splits it into two subsets while preserving the distribution of a specified grouping variable.

Usage

```
stratified_split(set, group, id, prop = 0.7)
```

Arguments

set	A data frame to be split into subsets.
group	A character string specifying the name of the outcome variable in set.
id	A character string specifying the name of the (categorical) stratum variable in set.
prop	A numeric value between 0 and 1 indicating the proportion of rows to include in the first subset. Defaults to 0.7.

Value

A list with two elements:

set1 The first subset of the dataset, containing approximately prop proportion of the rows, stratified by the grouping variable.

set2 The second subset of the dataset, containing the remaining rows.

Examples

```
# Load a dataset
data(altmejd)

# Perform a 70-30 stratified split by
split <- stratified_split(altmejd, group = "replicate", id = "eid", prop = 0.7)

# Access the subsets
set1 <- split$set1
set2 <- split$set2
```

testAda	<i>Generate Predictions or Retrodictions From a Set of (Adaptively) Boosted Trees and Model Weights</i>
---------	---

Description

This function implements the testing phase of the AdaBoost algorithm. It extracts the adaptively boosted weak learners (e.g. classification stumps) and their corresponding weights to combine them into the weighted sum

$$H(\mathbf{x}) = \sum_{i=1}^T a_i h_i(\mathbf{x}),$$

which yields AdaBoost's predictions or retrodictions on a given set.

Usage

```
testAda(fit, data, input_checks = TRUE, verbose = TRUE)
```

Arguments

fit	A model fitted with <code>trainAda</code> . This is a list of boosted trees and their weights. More specifically, each element of the list contains a weak learner (e.g., a decision stump) and its corresponding weight.
data	A data frame containing the test set for predictions. The structure should match the training set used to generate <code>h</code> .
input_checks	A logical value indicating whether to perform input validation checks. Defaults to TRUE.
verbose	A logical value specifying whether to display progress messages and animations. Defaults to TRUE.

Details

The function proceeds as follows:

1. If `input_checks` is TRUE, basic input validation is performed to ensure that `fit` is a valid AdaBoost model and `data` is appropriate for predictions.
2. Progress messages and animations are shown if `verbose` is TRUE.
3. Boosted weak learners (`h`) and their weights (`a`) are extracted from `fit`.
4. Predictions or retrodictions are generated for each weak learner using the test set.
5. Individual predictions are combined using the weights from the AdaBoost model to produce the final predictions.

Value

A numeric vector containing the final predictions from the AdaBoost model. The predictions are in the form of -1 or 1, corresponding to binary classification outcomes.

Examples

```
# Example usage:
# Assume `fit` is a trained model and `test` is a data frame.
# ypred <- testAda(fit, test)
```

trainAda

Train a Set of Classification Trees Using AdaBoost

Description

The function trains a set of classification trees using AdaBoost. It is built on top of the `rpart` package, so the full range of tree hyperparameters can be used to fine-tune the trees (see, e.g., [rpart.control](#)). In addition, the implementation allows customization of the number of trees and the learning rate. It also provides verbose output to track the training progress.

Usage

```
trainAda(formula, data, T, eta, treehypar, input_checks = TRUE, verbose = TRUE)
```

Arguments

formula	A formula specifying the relationship between the outcome and the predictors: outcome ~ predictors. The predictors should be included as additive terms (e.g., X1 + X2 + . . .), Interactions are not supported (see the formula argument in rpart).
data	A data frame containing the variables in the model.
T	An integer specifying the number of trees.
eta	A numeric value representing the learning rate of the algorithm.
treehypar	A list of control parameters for decision trees (passed to rpart.control).
input_checks	A logical value indicating whether to perform input validation checks. Defaults to TRUE.
verbose	A logical value indicating whether to display verbose output during the training process. Defaults to TRUE.

Details

The function implements the AdaBoost algorithm with a specified number of iterations (T). It initializes observation weights, trains a sequence of decision trees, and updates the weights at each iteration based on prediction errors. A final ensemble of weak learners is produced.

Key steps in the algorithm:

1. Initialize observation weights.
2. Train a decision tree using the current weights.
3. Compute the weighted classification error and update the observation weights.
4. Store the weak learner and its associated weight.

Value

A list containing the trained weak learners (h) and their associated weights (a). Additional attributes may be included for model tracking purposes.

Examples

```
## Not run:
# Example usage:
library(rpart)
data(iris)
# Prepare binary classification data
irisbin <- iris[iris$Species != "setosa", ]
irisbin$Species <- factor(irisbin$Species)

# Set tree hyperparameters
treehypar <- rpart::rpart.control(maxdepth = 1, cp = 0)

# Train AdaBoost model
model <- trainAda(Species ~ ., data = irisbin, T = 10, eta = 1,
  treehypar = treehypar)

## End(Not run)
```

Index

* datasets

- altmejd, [2](#)
- altmejd723, [2](#)
- ssrp723, [7](#)

altmejd, [2](#), [2](#), [7](#)
altmejd723, [2](#)

check_df (checks), [3](#)
check_eta (checks), [3](#)
check_length (checks), [3](#)
check_list (checks), [3](#)
check_numeric (checks), [3](#)
check_prop (checks), [3](#)
check_train (checks), [3](#)
checks, [3](#)
color_message (feedback), [5](#)

feedback, [5](#)
formula, [10](#)

random_split, [6](#)
rpart, [10](#)
rpart.control, [9](#), [10](#)

ssrp723, [7](#)
stratified_split, [7](#)

testAda, [8](#)
trainAda, [9](#), [9](#)

walking_colordots (feedback), [5](#)
walking_dots (feedback), [5](#)